Apollo MxFE® X-grade to B-grade API changes

The Apollo MxFE product is offered as engineering grade for now and branded as X-grades that utilize engineering keys with encrypted firmware. As Apollo MxFE products are released to the market, the offered product will be branded as B-grades and the associated firmware will utilize different encrypted images with production keys.

	X-Grade Devices	B-Grade Devices
Purpose	Pre-released materials for evaluation.	Released materials for production.
API	Supported by all API versions.	Supported by API v0.4.0 or newer versions only.
Firmware	Supported by all firmware versions.	Supported by firmware 20241008.1.281 or newer versions only.
Device Profile	Supported by all device profile versions.	Supported by all device profile v9 or newer versions.
SDK	All SDKs older than SDK45	SDK45.1 or newer versions using device profile v9 or newer versions
Encryption Keys	Firmware is encrypted and signed with engineering keys.	Firmware is encrypted and signed with production keys.
Ordered Part number	AD9084XBPZ-MX-DF	AD9084BBPZ-MX-DF-SW1 AD9084BBPZ-MX-DF-SW3

The API version is critical to determine if upgrading is required.

For API versioned v0.3.39 or older, it only supports X-grades devices. These versions support:

1) Normal Boot (unencrypted firmware images)
2) Secure Boot for X-grade devices only (encrypted FW images with engineering keys)

For API versioned v0.4.0 or newer, it supports both X-grade and B-grade devices. The API package consists of:

1) Normal boot (unencrypted firmware images)
2) Secure boot for X-grade devices (encrypted firmware images with engineering keys)
3) Secure boot for B-grade devices (encrypted firmware images with production keys)

User Code Changes

First, the user must add four additional #defines to the top-level code so the API FW load function will know where to find the FW binary files.

File with changes:
./examples/ads10_apollo_ex_common/include/adi_ads10_apollo_ex_types.h

API v0.3.39 or older has this coding for reference:

```
// Unencrypted images for normal boot
#define FW_IMAGES_DIR           "../../examples/fw_images/b0/"
#define CORE_0_STD_FW_BIN       FW_IMAGES_DIR"APOLLO_FW_CPU0_B.bin"
#define CORE_1_STD_FW_BIN       FW_IMAGES_DIR"APOLLO_FW_CPU1_B.bin"

// Encrypted images with engineering keys for secure boot
#define SECR_BOOT_HDR_BIN
        FW_IMAGES_DIR"app_signed_encrypted_B/flash_image_0x01030000.bin"
#define CORE_0_TYE_FW_BIN
        FW_IMAGES_DIR"app_signed_encrypted_B/flash_image_0x20000000.bin"
#define CORE_1_TYE_FW_BIN
        FW_IMAGES_DIR"app_signed_encrypted_B/flash_image_0x02000000.bin"
#define TYE_OPER_FW_BIN
        FW_IMAGES_DIR"app_signed_encrypted_B/flash_image_0x21000000.bin"
```

API v0.4.0 or newer require additional #defines:

```
// Unencrypted images for normal boot
#define FW_IMAGES_DIR           "../../examples/fw_images/b0/"
#define CORE_0_STD_FW_BIN       FW_IMAGES_DIR"APOLLO_FW_CPU0_B.bin"
#define CORE_1_STD_FW_BIN       FW_IMAGES_DIR"APOLLO_FW_CPU1_B.bin"

// Encrypted images with engineering keys for secure boot
#define SECR_BOOT_HDR_BIN
        FW_IMAGES_DIR"app_signed_encrypted_B/flash_image_0x01030000.bin"
#define CORE_0_TYE_FW_BIN
        FW_IMAGES_DIR"app_signed_encrypted_B/flash_image_0x20000000.bin"
#define CORE_1_TYE_FW_BIN
        FW_IMAGES_DIR"app_signed_encrypted_B/flash_image_0x02000000.bin"
#define TYE_OPER_FW_BIN
        FW_IMAGES_DIR"app_signed_encrypted_B/flash_image_0x21000000.bin"

// Encrypted images with production keys for secure boot
#define PROD_SECR_BOOT_HDR_BIN
        FW_IMAGES_DIR"app_signed_encrypted_prod_B/flash_image_0x01030000.bin"
#define PROD_CORE_0_TYE_FW_BIN
        FW_IMAGES_DIR"app_signed_encrypted_prod_B/flash_image_0x20000000.bin"
#define PROD_CORE_1_TYE_FW_BIN
        FW_IMAGES_DIR"app_signed_encrypted_prod_B/flash_image_0x02000000.bin"
#define PROD_TYE_OPER_FW_BIN
        FW_IMAGES_DIR"app_signed_encrypted_prod_B/flash_image_0x21000000.bin"
```

API Function Changes

Second, the specific application function change occurs within the **tye_fw_load(…)** call. Code will be added to check device grade value and apply the appropriate FW image (X-grade or B-grade).

File with changes:
./examples/ads10_apollo_ex_common/src/adi_ads10_apollo_ex.c

API v0.3.39 or older has this coding for reference:

```
static int32_t tye_fw_load(adi_apollo_device_t *device, uint8_t scenario)
{
    int32_t err;

    char *secr_hdr_filename = SECR_BOOT_HDR_BIN;
    char *core0_fw_filename = CORE_0_TYE_FW_BIN;
    char *core1_fw_filename = CORE_1_TYE_FW_BIN;
    char *oper_fw_filename  = TYE_OPER_FW_BIN;

    ...
    // Load the firmware header to memory.
    ...
    // Scenarios 2 and 4 app_pack contains the core 0 firmware image.
    ...
    // Scenarios 3 and 4 app_pack contains the core 1 firmware image.
    ...
    // Load the operation firmware image to memory.
```

API v0.4.0 or newer has the additional code to support B-grade devices.

```
static int32_t tye_fw_load(adi_apollo_device_t *device, uint8_t scenario)
{
    int32_t err;
    uint8_t si_grade;

    char *secr_hdr_filename = NULL;
    char *core0_fw_filename = NULL;
    char *core1_fw_filename = NULL;
    char *oper_fw_filename  = NULL;

    ...

    err = adi_apollo_device_si_grade_get(device, &si_grade);
    ADI_CMS_ERROR_RETURN(err);
    printf("Apollo Silicon Grade = %02d\n", si_grade);

    secr_hdr_filename = (si_grade == 0) ? SECR_BOOT_HDR_BIN :
                         PROD_SECR_BOOT_HDR_BIN;
    core0_fw_filename = (si_grade == 0) ? CORE_0_TYE_FW_BIN :
                         PROD_CORE_0_TYE_FW_BIN;
    core1_fw_filename = (si_grade == 0) ? CORE_1_TYE_FW_BIN :
                         PROD_CORE_1_TYE_FW_BIN;
    oper_fw_filename = (si_grade == 0) ? TYE_OPER_FW_BIN :
                        PROD_TYE_OPER_FW_BIN;

     ...
     // Load the firmware header to memory.
     ...
     // Scenarios 2 and 4 app_pack contains the core 0 firmware image.
     ...
     // Scenarios 3 and 4 app_pack contains the core 1 firmware image.
     ...
     // Load the operation firmware image to memory.
     ...
}
```

Summary:

For a user's software to support production grade Apollo MxFE® devices, changes are required in the user code and additional define statements are used to properly boot up the device. The released devices, called B-grades, have different identifiers that the firmware needs to use for proper device boot up. This application note highlights the two sections of code that are required to be added for successful boot up of B-grade devices. For API versions v0.4.0 or newer, backwards compatibility is retained so that one set of source code can work on booting up either X-grade or B-grade devices.